Islamic University of Technology (IUT)
Department of Computer Science & Engineering



Software Design Report

Squiggle

Real-Time Multiplayer Drawing & Guessing Game

SWE 4404

Group 7

Group Members

	Name	Student ID
1	Ashfaque Sadad	230042102
2	MD Zahin Uddin Nirob	230042160
3	Sheikh Alimun Alahi	230042115

Contents

1 Project Overview **1**

2 Class Diagram **2**
2.1 Core Classes & Entity Descriptions 2
2.2 Class Relationships and Cardinalities 3

3 Three-Layer Architecture Diagram **4**
3.1 Presentation Layer (React Frontend) 4
3.2 Business Logic Layer (Node/Express Server) 4
3.3 Data Access & Storage Layer (Mongoose & MongoDB) 5

4 Software Requirements Specification (SRS) **6**

1 Project Overview

Squiggle is a real-time, multiplayer drawing and guessing game engineered to foster user engagement through creativity and seamless social interaction. While many drawing games focus heavily on competitive pressure, Squiggle is designed to prioritize fun and accessibility, ensuring an enjoyable experience for a diverse audience of players. The system supports both registered and guest players, real-time collaborative drawing and chat-based guessing over Socket.IO, AI-assisted word generation via the Gemini API, a coin/diamond reward economy, and a range of power-ups and alternate game modes such as a Wordle-inspired mode.

2 Class Diagram

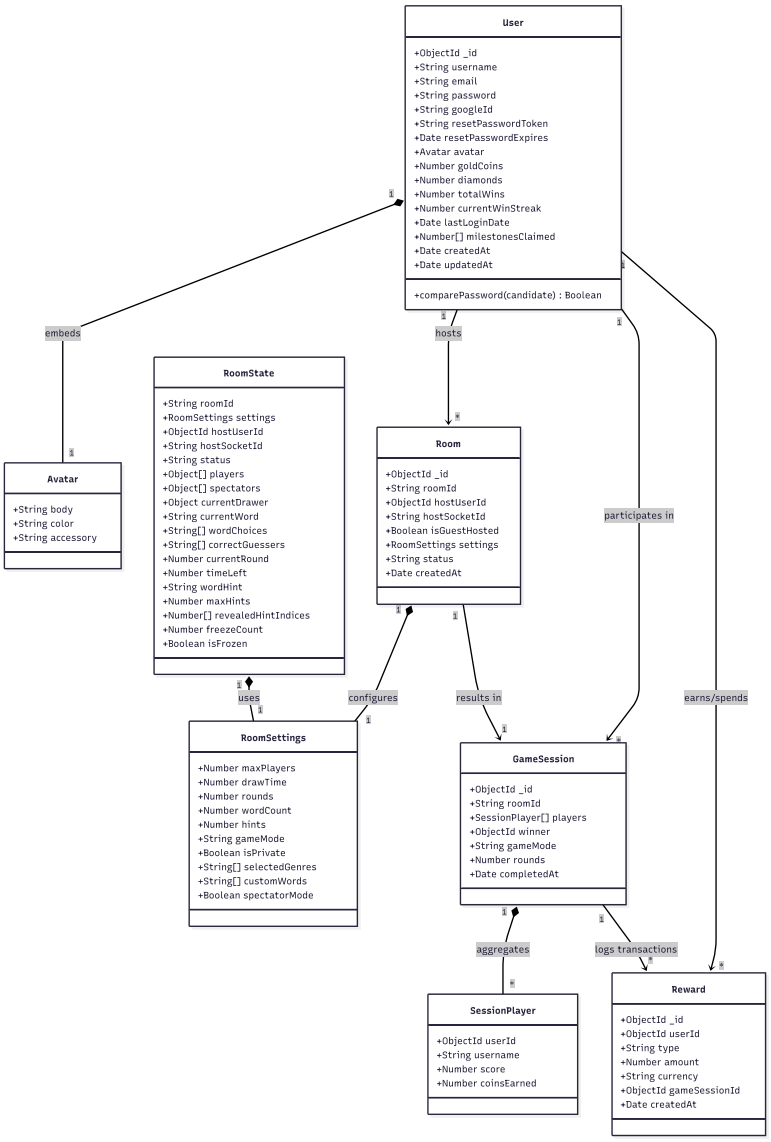


Figure 1: Class Diagram of the Squiggle System

This class diagram represents the structure of the system by illustrating its classes, attributes, methods, and the relationships among them. It bridges the persistent database layer with the real-time execution engine.

2.1 Core Classes & Entity Descriptions

User: Represents registered player accounts containing user credentials, current balances (`goldCoins`, `diamonds`), competitive stats (`totalWins`, `currentWinStreak`), and password authentication methods (`comparePassword`).

Avatar: A value object class embedded inside `User` defining the visual customization parameters (body type, hex color, and accessory) of a user's avatar profile.

Room: Represents room meta-configurations saved in MongoDB. It stores the unique join code (`roomId`), host descriptors, active phase status, and custom limits.

RoomSettings: Configures game options (maximum player limit, drawing round countdowns, chosen categories/genres, custom word pools, privacy status, and spectator permissions).

GameSession: Formulates matching summaries compiled on final game completion.

SessionPlayer: Records player scores and rewards specifically earned during a match instance.

Reward: A ledger system documenting transaction instances representing currency updates (e.g., milestone achievements, streak bonuses, daily rewards, power-up costs).

RoomState: The runtime logic class executing live match parameters in-memory. It maintains connected players, round configurations, timing clocks, drawn assets, secret words, guesses, hints, and active power-ups (e.g., freeze timer).

2.2 Class Relationships and Cardinalities

- **Composition**
  - `User` has a 1-to-1 composition with `Avatar` (`User *-- Avatar`). An Avatar cannot exist independently outside of a `User`.
  - `Room` has a 1-to-1 composition with `RoomSettings` (`Room *-- RoomSettings`).
  - `GameSession` has a 1-to-*many* composition with `SessionPlayer` (`GameSession *-- SessionPlayer`).
- **Association (Directed arrow lines):**
  - **User to Room (1-to-*many*):** A User can host multiple rooms (`User --> Room`).
  - **User to Reward (1-to-*many*):** A User can earn or spend coins through multiple currency updates (`User --> Reward`).
  - **User to GameSession (1-to-*many*):** A User participates in multiple game sessions over their lifetime (`User --> GameSession`).
  - **Room to GameSession (1-to-1):** A single Room results in exactly one GameSession summary report (`Room --> GameSession`).
  - **GameSession to Reward (1-to-*many*):** A session generates multiple transaction logs for rewards (`GameSession --> Reward`).

3 Three-Layer Architecture Diagram

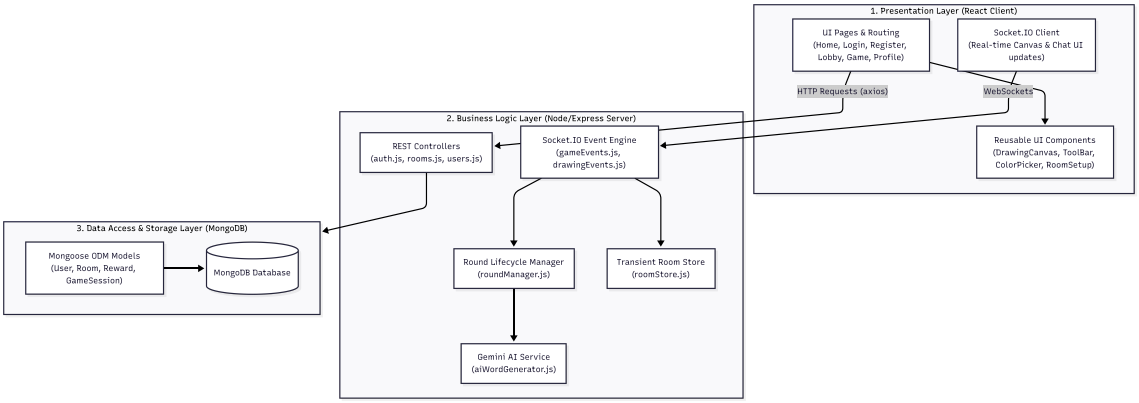


Figure 2: Three-Layer Architecture of the Squiggle System

This diagram displays the software architecture, showing how responsibilities are cleanly divided into three decoupled tiers to promote modularity, security, and performance.

3.1 Presentation Layer (React Frontend)

Responsible for capturing user inputs and rendering visual information. It comprises React router pages (`Home`, `LobbyPage`, `GamePage`), modular UI components (canvas coordinates, styling toolbar, settings configuration), and the `Socket.IO Client`. The client-side layer interacts with the backend using two protocols:

- **HTTP Requests:** Traditional REST requests via Axios for non-realtime operations (register, login, loading user profile).
- **WebSocket Streams:** Instantaneous streams mapping live drawing strokes, guessing text, and clock timer adjustments.

3.2 Business Logic Layer (Node/Express Server)

This is the core application logic engine. It processes inputs, makes calculations, and controls the flow:

- **REST Controllers (`auth.js`, `rooms.js`, `users.js`):** Handle authentication validation, daily rewards calculation, profile updates, and database queries.
- **Socket.IO Events Engine (`gameEvents.js`, `drawingEvents.js`):** Synchronizes drawing coordinates and routes chat message guess validations.
- **Round Manager (`roundManager.js`):** Orchestrates gameplay phases (Word Selection, Drawing, Guessing, Intermission) and triggers automatic events (like progressive hints or drawing timeout).
- **Transient Room Store (`roomStore.js`):** An in-memory Map structure managing game states during live gameplay to prevent database write bottlenecks.

- **AI Service (`aiWordGenerator.js`):** Connects to the Google Gemini API to fetch context-specific game words for selected genres, falling back to static lists on network failure.

3.3 Data Access & Storage Layer (Mongoose & MongoDB)

Manages long-term database persistence. It consists of the Mongoose schemas acting as the Object-Document Mapper (ODM) mapping structural definitions of user profiles, permanent room configurations, transaction ledger records, and post-game history logs. Data operations from the business layer are validated here before writing to the database collection tables.

4 Software Requirements Specification (SRS)

The following table summarizes the functional requirements of Squiggle. For the full breakdown – including user stories, priority levels, and implementation status for each item – see the detailed spreadsheet linked below.

View Full SRS Spreadsheet

	Functional Requirement	Acceptance Criteria
1	FR_1: Allow users to register with a unique username and password.	• Username unique (3–20 characters); password securely hashed • Duplicate usernames rejected with a clear error • Successful registration returns a JWT; new users start with 5 diamonds
2	FR_2: Allow registered users to log in.	• Correct credentials log in; invalid credentials show a clear error • Returns a JWT valid for 7 days plus full profile data
3	FR_3: Reward 2 gold coins on the first login of each day.	• Bonus awarded once per day, checked against last login date • Balance updates immediately; no bonus if already claimed today
4	FR_4: Allow guest play without registering.	• Guests can join and play but earn no permanent coins/diamonds • Milestone rewards disabled for games with guest players
5	FR_5: Allow creating a public or private game room with custom settings.	• Host configures player count (2–20), draw time (15–240s), rounds, hints (0–5), and mode • Unique 6-character room code generated; private rooms require the code to join
6	FR_6: Generate AI-assisted word choices by genre.	• 3 genre-relevant word choices offered per round, avoiding repeats within a session • Falls back to a static word list if the AI call fails; all 10 genres supported
7	FR_7: Provide a real-time drawing canvas.	• Brushes, adjustable eraser, undo/redo, clear, shapes, color picker, and fill tool • All drawing actions sync in real time across clients

	Functional Requirement	Acceptance Criteria
8	FR_8: Synchronize drawing actions in real time across clients.	• Strokes sent as vector coordinates with minimal latency • Reconnecting players receive the current canvas state
9	FR_9: Provide live chat for submitting guesses.	• Correct guesses auto-detected; guesser notified instantly and blocked from further guesses • System messages announce round results and player join/leave events
10	FR_10: Calculate and distribute coin/diamond rewards by performance.	• Coins for successful drawing and wins; diamonds every 3-win streak • Milestone bonuses at 10 and 20 wins (disabled with guests present); all rewards logged
11	FR_11: Maintain a leaderboard of the top 20 players.	• Ranked by total wins, then coins; shows wins, coins, diamonds, and streak • Updates after every completed game
12	FR_12: Maintain a reward history log.	• Shows the last 50 transactions (type, amount, currency, date) • Visible only to the account owner
13	FR_13: Allow spending coins/diamonds on power-ups during a game.	• Balance validated before purchase; deducted immediately and logged • Power-ups disabled in Custom Word and Genre modes
14	FR_14: Allow joining a room as a spectator, with a separate chat.	• Full rooms auto-assign spectators; otherwise players choose to spectate • Spectators view canvas/chat read-only in a separate, clearly labeled channel; no voice access
15	FR_15: Provide real-time peer-to-peer voice chat.	• WebRTC voice chat for players only, not spectators • Mute/unmute, push-to-talk, and per-player volume control
16	FR_16: Provide an in-game reference image search for the drawer.	• Search results shown only to the drawer, hidden from other players • Free to use, available in all game modes

	Functional Requirement	Acceptance Criteria
17	FR_17: Provide a Wordle-inspired game mode.	• Guesses 1–2 are blind (60s each) with green/yellow/gray letter feedback • Drawing begins at guess 3 (30s each); points decrease per guess, with drawer bonus points based on how many players succeed • Correct guessers are blocked from typing the answer in chat to prevent spoilers
18	FR_18: Provide a Hint Steal power-up.	• Reveals one missing letter of the current word • Cost scales from 100 coins/1 diamond up to 300 coins/2 diamonds as fewer letters remain; disabled in Custom Word and Genre modes
19	FR_22: Implement a vote-kick mechanism.	• Requires unanimous agreement; not usable against the host • Kicked player loses 50 coins (or owes it from future earnings) and cannot rejoin the room
20	FR_23: Display a live in-game leaderboard.	• Updates after each round with player name, score, and rank • Winner announced at the end; ties broken by earliest correct guess
21	FR_24: Award points based on guessing speed.	• Faster correct guesses earn more points, decreasing for later guessers • Drawer earns bonus points when more players guess correctly; applies to Standard and Wordle modes